

AFRILANG Stdlib

std/c/cryptsha256

Bibliothèque AFRILANG `std/c/cryptsha256` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : sha2Len, sha2

```
`import "std/c/cryptsha256" · `use cryptsha256`
```

- `sha2Len(s text) ? number`
- `sha2Upper(s text) ? text`
- `sha2Lower(s text) ? text`
- `sha2Trim(s text) ? text`
- `sha2Split(s text, delim text) ? list text`
- `sha2Join(parts list text, delim text) ? text`
- `sha2Replace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/cryptsha256` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : sha2Len, sha2

```
`import "std/c/cryptsha256" . `use cryptsha256`  
- `sha2Len(s text) ? number`  
- `sha2Upper(s text) ? text`  
- `sha2Lower(s text) ? text`  
- `sha2Trim(s text) ? text`  
- `sha2Split(s text, delim text) ? list text`  
- `sha2Join(parts list text, delim text) ? text`  
- `sha2Replace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/cryptsha256"  
use cryptsha256
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? sha2Len(s text) ? number  
? sha2Upper(s text) ? text  
? sha2Lower(s text) ? text  
? sha2Trim(s text) ? text  
? sha2Split(s text, delim text) ? list  
? sha2Join(parts list text, delim text) ? text  
? sha2Replace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/cryptsha256"  
use cryptsha256  
  
create result = sha2Len(/* args */)   
say result  
  
create result = sha2Upper(/* args */)   
say result  
  
create result = sha2Lower(/* args */)   
say result  
  
create result = sha2Trim(/* args */)   
say result  
  
create result = sha2Split(/* args */)   
say result  
  
create result = sha2Join(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/cryptsha256"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module cryptsha256
function sha2Len(s text) returns number
end
function sha2Upper(s text) returns text
end
function sha2Lower(s text) returns text
end
function sha2Trim(s text) returns text
end
function sha2Split(s text, delim text) returns list text
end
function sha2Join(parts list text, delim text) returns text
end
function sha2Replace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/cryptsha256` (tier complex, category complex). Exports 8 function(s): sha2Len, sha2Upper, sha2Lo

```
`import "std/c/cryptsha256" . `use cryptsha256`  
- `sha2Len(s text) ? number`  
- `sha2Upper(s text) ? text`  
- `sha2Lower(s text) ? text`  
- `sha2Trim(s text) ? text`  
- `sha2Split(s text, delim text) ? list text`  
- `sha2Join(parts list text, delim text) ? text`  
- `sha2Replace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/cryptsha256"  
use cryptsha256
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? sha2Len(s text) ? number  
? sha2Upper(s text) ? text  
? sha2Lower(s text) ? text  
? sha2Trim(s text) ? text  
? sha2Split(s text, delim text) ? list  
? sha2Join(parts list text, delim text) ? text  
? sha2Replace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/cryptsha256"  
use cryptsha256  
  
create result = sha2Len(/* args */)   
say result  
  
create result = sha2Upper(/* args */)   
say result  
  
create result = sha2Lower(/* args */)   
say result  
  
create result = sha2Trim(/* args */)   
say result  
  
create result = sha2Split(/* args */)   
say result  
  
create result = sha2Join(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/cryptsha256"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module cryptsha256
function sha2Len(s text) returns number
end
function sha2Upper(s text) returns text
end
function sha2Lower(s text) returns text
end
function sha2Trim(s text) returns text
end
function sha2Split(s text, delim text) returns list text
end
function sha2Join(parts list text, delim text) returns text
end
function sha2Replace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```